Flutter Hydra Package Integration

This document helps you integrate the `capillary_hydra` Flutter plugin package, published on pub.dev, in your Flutter apps.

Prerequisites

Before you get started, you need to register your app with the Capillary apps team. For registration, you need to provide these credentials:

- Bundle id for your iOS App
- ApplicationId for your Android App
- App signature (SHA256) from your Android app signing report

When you provide these credentials to our apps team, they will register the app with our system and provide you these details:

- Account Id
- Base url (End point)
- SSL public key

If you plan to use the Anonymous/Visitor Flow, you will additionally need this detail from the apps team:

- Org Id

If you plan to use Gamification (banners, embeds, loyalty widgets), you will additionally need these details from the apps team:

- Gamification Key
- Gamification Region
- Gamification Base Url

Getting Started

1. It's mandatory to get Account Id, SSL public key, Base Url (End point), and Remote log level from a remote configuration tool (CMS tool). By default the SDK fetches these from Firebase Remote Config — add these 4 parameters in the Firebase console with these exact names:

   - `hydra_account_id`
   - `hydra_base_url`
   - `hydra_ssl_public_key`
   - `hydra_remote_log_level`

   If you also want to use the Anonymous/Visitor Flow, add this parameter:

   - `hydra_org_id`

   If you also want to use Gamification, add these 3 parameters:

   - `hydra_gamification_key`
   - `hydra_gamification_region`
   - `hydra_gamification_base_url`

   If you're using a multi-Hydra account setup, set the most-default values in the keys above, and for remaining Hydra accounts define new keys with a prefix — e.g. for an India account: `ind_hydra_account_id`, `ind_hydra_base_url`, `ind_hydra_ssl_public_key`, `ind_hydra_remote_log_level`, and similarly prefixed `ind_hydra_org_id` / `ind_hydra_gamification_key` / `ind_hydra_gamification_region` / `ind_hydra_gamification_base_url` if applicable. You pass these custom key names via `HydraCore.setFirebaseRemoteConfigKeys` when you want to switch to a custom Hydra account from Dart.

If you don't want to use Firebase Remote Config at all — either a different CMS, or passing values directly — you can fetch these values yourself and pass them to the SDK directly, either natively (see Advanced Android Setup and Advanced iOS Setup) or from Dart via `HydraCore.setHydraAccountConfig`.

2. There's an example app on the `capillary_hydra` pub.dev page you can refer to while integrating.

Setup for Flutter

1. Add these 2 dependencies to your app's `pubspec.yaml`. We'll announce when a new version is available:

```
dependencies:
  capillary_hydra: ^1.1.1-beta.6
  capillary_hydra_platform_interface: ^1.1.1-beta.6
```

Or run:

```
flutter pub add capillary_hydra:^1.1.1-beta.6 capillary_hydra_platform_interface:^1.1.1-beta.6
```

2. Add the config file as an asset:

   - Create a folder named `capillary` at the root of your Flutter project.
   - Create a file named `hydra-config.json` inside `capillary/` with these contents:

```
{
  "console_log_level": 4,
  "enable_in_app_messaging": false,
  "enable_inbox_api": false,
  "enable_inbox_ui": false,
  "use_default_inbox_ui_config": true,
  "max_notification_count": 200,
  "hydra_notification_color": "#FF03DAC5",
  "enable_in_app_messaging_in_flutter": false
}
```

   - Set the values as needed — see hydra-config.json below.
   - In your `pubspec.yaml`, declare the folder (and its `icons` subfolder, once you add icon assets per later sections) as assets:

```
flutter:
  assets:
    - capillary/
    - capillary/icons/
```

3. Run:

```
flutter pub get
```

4. Call `HydraInitializer.initialize()` in `main()`, before `runApp`:

```dart
import 'package:capillary_hydra/hydra.dart';
import 'package:flutter/material.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  HydraInitializer.initialize();
  runApp(const MyApp());
}
```

---

hydra-config.json

Key	Type	Default	Platform	Description
`console_log_level`	number	4	Android & iOS	Console log level (0–5). 0=off, 1=error, 2=warn, 3=info, 4=debug, 5=verbose.
`enable_in_app_messaging`	boolean	false	Android & iOS	Set to `true` to enable in-app messaging. Requires the in-app placeholder asset — see InApp Messaging.
`enable_in_app_messaging_in_flutter`	boolean	false	**Android only**	See InApp Messaging — **currently non-functional, do not enable in production**.
`enable_inbox_api`	boolean	false	Android & iOS	Set to `true` to enable the Inbox API — see Inbox API.
`enable_inbox_ui`	boolean	false	Android & iOS	Set to `true` to use the built-in notification center — see Inbox UI.
`use_default_inbox_ui_config`	boolean	true	**iOS only**	Use the SDK default notification center layout.
`max_notification_count`	number	200	Android & iOS	Maximum number of notifications kept in the inbox / notification center.
`hydra_notification_color`	string	–	**Android only**	Notification tint color (e.g. `"#FF03DAC5"`) — see Push Notification.

You can also set a remote log level (0–5, same scale) via `hydra_remote_log_level` in Firebase Remote Config, as described in Getting Started.

Setup for Android

1. **Set the Application class in `AndroidManifest.xml`.** Flutter doesn't use a custom Application class by default, but the Hydra SDK needs to run initialization in one. If you don't already have your own Application class, point directly at the one we provide:

   ```xml
   <application
       android:name="com.capillary.plugins.hydra.HydraApplication"
       ...>
   ```

   If you already have your own Application class, or want to use a CMS other than Firebase Remote Config, see Advanced Android Setup instead.

2. **Set an AppCompat theme.** The Hydra plugin requires your application theme to derive from AppCompat:

   ```xml
   <application
       ...
       android:theme="@style/Theme.AppCompat.DayNight.NoActionBar">
   ```

Setup for iOS

If Swift Package Manager is enabled in your project, skip steps 1–2 below.

1. In `ios/Podfile`, add this line at the top:

```
require_relative './.symlinks/plugins/capillary_hydra_ios/script/HydraPods.rb'
```

2. In your `Runner` target, add `hydra_pods`:

```
target 'Runner' do
  use_frameworks!

  hydra_pods
  flutter_install_all_ios_pods File.dirname(File.realpath(__FILE__))
  target 'RunnerTests' do
    inherit! :search_paths
  end
end
```

3. In your `ios` folder, run:

```
pod install
```

4. If you want to use a CMS other than Firebase Remote Config, see Advanced iOS Setup.

Push Notification

1. Register your app on Firebase by following this link. Download `google-services.json` and `GoogleService-Info.plist`. Don't make any code changes suggested while generating these files.
2. **For iOS:**
    - Add `GoogleService-Info.plist` to your iOS project using Xcode.
    - Make sure Push Notification capability is added to your app identifier on developer.apple.com.
    - In Xcode project settings for your app target, add these capabilities under Signing & Capabilities: Push Notification, and Background Modes with Background fetch and Remote notifications enabled.
    - **Rich notifications are mandatory.** Set up a Notification Service Extension and wire it to Hydra:
        - Add a Notification Service Extension: in Xcode, go to your project settings and add a new target for this. Xcode will append the extension's product name to your main target's bundle id to derive its bundle id — add that new bundle id on developer.apple.com.
        - Add **App Groups** capability in Signing & Capabilities for both your main target and this extension target, using the same group in both, in the form `group.<runner_target_bundle_id>.hydra`. Create this App Group on developer.apple.com for both identifiers.
        - If Swift Package Manager is enabled for your project, skip this step. Otherwise, in `ios/Podfile`, add `extension_pods` inside your new extension target (e.g. `HydraRichNotification`):

```
target "HydraRichNotification" do
  use_frameworks!
  extension_pods
end
```

Then run `pod install` again as in step 3 of Setup for iOS.

- Replace the entire contents of `NotificationService.swift` with:

```
import UserNotifications
import HydraPushNotificationServiceExtension

class NotificationService: UNNotificationServiceExtension {
    let service: HydraRemoteNotificationService = .init()

    override func didReceive(_ request: UNNotificationRequest, withContentHandler contentHandler:
@escaping (UNNotificationContent) -> Void) {
        service.didReceive(request, withContentHandler: contentHandler)
    }

    override func serviceExtensionTimeWillExpire() {
        service.serviceExtensionTimeWillExpire()
    }
}
```

3. **For Android:**
   - Add `google-services.json` to the `android/app` folder.
   - Declare the Google Services plugin in your root `plugins` block:

     `android/settings.gradle.kts` (Kotlin):

     ```
     id("com.google.gms.google-services") version "4.4.2" apply false
     ```

     `android/settings.gradle` (Groovy):

     ```
     id 'com.google.gms.google-services' version '4.4.2' apply false
     ```

   - Apply the plugin in your app's `plugins` block:

     `android/app/build.gradle.kts` (Kotlin):

     ```
     id("com.google.gms.google-services")
     ```

     `android/app/build.gradle` (Groovy):

     ```
     id 'com.google.gms.google-services'
     ```

4. Set `hydra_notification_color` in `capillary/hydra-config.json` (see hydra-config.json).
5. Create a folder named `icons` inside `capillary/` if it doesn't already exist, and add an image named `hydra_notification_small_icon` (png, jpg, or jpeg — png recommended). This is the small icon shown in the notification drawer.

InApp Messaging

1. Set `enable_in_app_messaging` to `true` in `hydra-config.json`.
2. Add an image named `hydra_in_app_placeholder` in `capillary/icons`. Can be png, jpg, or jpeg.
3. **For iOS:**
    - In `AppDelegate.swift`, add this import at the top:

      ```
      import capillary_hydra_ios
      ```

    - Add this function:

      ```
      override func application(_ application: UIApplication, didReceiveRemoteNotification userInfo:
      [AnyHashable: Any], fetchCompletionHandler completionHandler: @escaping (UIBackgroundFetchResult) ->
      Void) {
          HydraNativeAPI.shared.application(application, didReceiveRemoteNotification: userInfo,
      fetchCompletionHandler: completionHandler)
      }
      ```

Inbox API

1. Set `enable_inbox_api` to `true` in `hydra-config.json`.
2. To use this on iOS, the Notification Service Extension must be set up — see the rich notifications part of step 2 in Push Notification.

Inbox UI

If you want to use the notification center provided by the Hydra SDK, enable this.

1. Enable Inbox API — see Inbox API.
2. Set `enable_inbox_ui` to `true` in `hydra-config.json`.
3. Add an image named `hydra_inbox_ui_placeholder` in `capillary/icons`. Can be png, jpg, or jpeg.

Advanced Android Setup

If you already have your own class that extends `Application`, make sure it extends `HydraApplication` instead, and update `AndroidManifest.xml` to point at your class's fully-qualified name (package + class name), as in step 1 of Setup for Android. The automatic default-key initialization only applies when the Application class is exactly `HydraApplication` — a subclass must call one of the initializers below itself, immediately after `super.onCreate()`.

There are 3 cases:

1. Your own Application class, default Firebase Remote Config keys — call `initializeUsingFirebaseRemoteConfig` with no keys argument:

```
import com.capillary.plugins.hydra.HydraApplication
import com.capillary.plugins.hydra.nativeapis.HydraInitializer

class ExampleApplication : HydraApplication() {
    override fun onCreate() {
        super.onCreate()
        HydraInitializer.initializeUsingFirebaseRemoteConfig(this)
        // ...
    }
}
```

2. Custom Firebase Remote Config key names (e.g. for a multi-account setup) — pass a `HydraFirebaseRemoteConfigKeys`:

```
import com.capillary.plugins.hydra.HydraApplication
import com.capillary.plugins.hydra.nativeapis.HydraInitializer
import com.capillary.plugins.hydra.nativeapis.HydraFirebaseRemoteConfigKeys

class ExampleApplication : HydraApplication() {
    override fun onCreate() {
        super.onCreate()
        val keys = HydraFirebaseRemoteConfigKeys(
            "ind_hydra_account_id",
            "ind_hydra_base_url",
            "ind_hydra_ssl_public_key",
            "ind_hydra_remote_log_level",
            "ind_hydra_org_id",
            "ind_hydra_gamification_key",
            "ind_hydra_gamification_region",
            "ind_hydra_gamification_base_url"
        )
        HydraInitializer.initializeUsingFirebaseRemoteConfig(this, keys)
        // ...
    }
}
```

3. Pass values directly (a CMS other than Firebase Remote Config, or hardcoded values) — build a `HydraRemoteConfig` via its `Builder` and pass it to `HydraInitializer.initialize` instead. `accountId`, `baseURL`, and `sslPublicKey` are required; `remoteDebugLevel` defaults to `0`; `orgId` (Anonymous/Visitor Flow) and `gamificationKey` / `gamificationRegion` / `gamificationBaseUrl` (Gamification) are optional:

```
import com.capillary.plugins.hydra.HydraApplication
import com.capillary.plugins.hydra.nativeapis.HydraInitializer
import com.capillary.plugins.hydra.nativeapis.HydraRemoteConfig

class ExampleApplication : HydraApplication() {
    override fun onCreate() {
        super.onCreate()
        val hydraRemoteConfig = HydraRemoteConfig.Builder(
            "your_account_id", "your_base_url", "your_ssl_public_key", 4 // LogType.DEBUG
        )
            .orgId("your_org_id")
            .gamificationKey("your_gamification_key")
            .gamificationRegion("your_gamification_region")
            .gamificationBaseUrl("your_gamification_base_url")
            .build()
        HydraInitializer.initialize(this, hydraRemoteConfig)
        // ...
    }
}
```

Advanced iOS Setup

By default the plugin initializes the Hydra SDK automatically using the default Firebase Remote Config keys. To change that, initialize it yourself from `AppDelegate.swift`, after `GeneratedPluginRegistrant.register(with:)` and before `super.application(...)`.

1. Custom Firebase Remote Config key names (e.g. for a multi-account setup) — pass a `HydraFirebaseRemoteConfigKeys`. `keyForAccountId`, `keyForBaseUrl`, `keyForSsl`, and `keyForRemoteLogLevel` are required; the org id and gamification keys are optional:

```
import capillary_hydra_ios

override func application(
  _ application: UIApplication,
  didFinishLaunchingWithOptions launchOptions: [UIApplication.LaunchOptionsKey: Any]?
) -> Bool {
  GeneratedPluginRegistrant.register(with: self)
  let keys = HydraFirebaseRemoteConfigKeys(
      keyForAccountId: "ind_hydra_account_id",
      keyForBaseUrl: "ind_hydra_base_url",
      keyForSsl: "ind_hydra_ssl_public_key",
      keyForRemoteLogLevel: "ind_hydra_remote_log_level",
      keyForOrgId: "ind_hydra_org_id",
      keyForGamificationKey: "ind_hydra_gamification_key",
      keyForGamificationRegion: "ind_hydra_gamification_region",
      keyForGamificationBaseUrl: "ind_hydra_gamification_base_url"
  )
  HydraNativeAPI.shared.initApplicationWithFirebaseRemoteConfig(application, didFinishLaunchingWithOptions:
launchOptions, firebaseRemoteConfigKeys: keys)
  return super.application(application, didFinishLaunchingWithOptions: launchOptions)
}
```

2. Pass values directly (a CMS other than Firebase Remote Config, or hardcoded values) — pass a `HydraRemoteConfig`. `accountId`, `baseUrl`, and `sslPublicKey` are required; `orgId` (Anonymous/Visitor Flow) and the gamification values

(Gamification) are optional:

```
import capillary_hydra_ios

override func application(
  _ application: UIApplication,
  didFinishLaunchingWithOptions launchOptions: [UIApplication.LaunchOptionsKey: Any]?
) -> Bool {
  GeneratedPluginRegistrant.register(with: self)
  let remoteConfig = HydraRemoteConfig(
      accountId: "your_account_id",
      baseUrl: "your_base_url",
      sslPublicKey: "your_ssl_public_key",
      remoteLogLevel: 4, // Debug level
      orgId: "your_org_id",
      gamificationKey: "your_gamification_key",
      gamificationRegion: "your_gamification_region",
      gamificationBaseUrl: "your_gamification_base_url"
  )
  HydraNativeAPI.shared.initApplication(application, didFinishLaunchingWithOptions: launchOptions, hydraRemoteConfig:
remoteConfig!)
  return super.application(application, didFinishLaunchingWithOptions: launchOptions)
}
```

Flutter Implementation

1. Before using any APIs from `capillary_hydra`, make sure the Hydra SDK has initialized successfully. Implement a `HydraNativeEventHandler` and register it:

   ```
   HydraNativeEvents.addEventHandler(handler);
   ```

   You'll receive the initialization result via:

   ```
   void onHydraSdkInitializationResult({required bool success, String? error})
   ```

2. Use `Hydra.identify` to register/identify the user — pass a `HydraUserDetails` with a required `customerId`, which must match how it's configured on the Capillary CRM dashboard. Use `Hydra.track` to report custom user events, and `Hydra.reset` to reset/sign out the current user. `HydraCore` also has static functions for `updateUser`, `updateGeoConfig`, and `updatePushPreferences`.

3. `HydraPushNotification` has static functions for `requestPermission` and `getFCMToken`. Call `requestPermission` before you can receive push. Call `getFCMToken` to get the token on demand, or register an event handler to get it via `onTokenAvailable`.

4. The SDK stores received notifications in a local database — use `HydraInbox` for `getInboxMessages`, `getUnreadInboxMessagesCount`, `markInboxMessageAsRead`, `deleteInboxMessage`, and `deleteAllInboxMessages`.

5. To use the built-in notification center, use `HydraInboxUI`'s `showNotificationCenter` and `hideNotificationCenterIfShowing`.

6. `HydraNativeEvents` has `addEventHandler`, `removeEventHandler`, and `removeAllEventHandlers`. Beyond `onTokenAvailable` and `onHydraSdkInitializationResult`, `HydraNativeEventHandler` also has callbacks for `onHydraNotificationReceived`, `onHydraNotificationClicked`, `onHydraNotificationDismissed`, `onHydraNotificationShown`, and `onNotificationCenterDismissed`.

7. Console log level is set via `console_log_level` in `hydra-config.json` (0–5). Remote log level is set separately via `hydra_remote_log_level` in Firebase Remote Config (same 0–5 scale), as covered in Getting Started.

Anonymous / Visitor Flow

The SDK supports an anonymous "visitor" flow. A user can move through states: `none` → `visitor` (anonymous) → `merged` / `legacyRegistered` (identified) → `signedOut`. This flow requires the Org Id mentioned in Prerequisites, configured the same way as Account Id/Base Url/SSL public key (via Firebase Remote Config key `hydra_org_id`, or passed directly — see Advanced Android Setup / Advanced iOS Setup and HydraAccountConfig/HydraFirebaseRemoteConfigKeys for the Dart-side equivalents).

1. Use `Hydra.isVisitorFlowEnabled()` to check whether the visitor flow is enabled for your account.
2. Use `Hydra.getCurrentUserState()` to get the current `UserState`.
3. Use `Hydra.getVisitorDetails()` to get the current `HydraVisitorInfo` (`visitorId`).
4. Subscribe to visitor-flow lifecycle events on your `HydraNativeEventHandler`: `onConfigFetched`, `onLegacyFlowReady`, `onVisitorSessionReady`, and `onUserStateChanged`.

`Hydra.identify` / `track` / `reset` (see Flutter Implementation) are also used as part of this flow.

Gamification

Gamification lets you show Floating Button and Popup campaigns, and embed banner, embed, and loyalty widgets directly as Flutter widgets. It requires the Gamification Key, Gamification Region, and Gamification Base Url from Prerequisites, configured the same way as the base account details — see HydraAccountConfig/HydraFirebaseRemoteConfigKeys for setting them explicitly from Dart if you're on a multi-account setup.

1. **Screen tracking:** call `Hydra.gamificationScreenAppeared(screenName: ...)` when a gamification-related screen appears, and `Hydra.gamificationScreenDisappeared(screenName: ...)` when it disappears — pass the **same** `screenName` to both calls. Once a screen is reported, Floating Button, Popup, and Bottom Sheet campaigns targeted at that screen show automatically — no further implementation needed for these.

2. **Native widgets** — three Flutter widgets, backed by native `PlatformView`s:

    - `GamificationBannerView({ required String bannerId, void Function(double height)? onHeightChanged })`
    - `GamificationEmbedView({ required String embedId, void Function(double height)? onHeightChanged })`
    - `GamificationLoyaltyView({ required String loyaltyId })` — fills whatever size it's given and never reports a height change; size it explicitly (e.g. `SizedBox` / `Expanded`). **Requires calling `Hydra.setCapToken(token: ...)` first**, before mounting — the native SDK cannot bind the Loyalty view without a cap token already set.

    Banner and embed widgets report their content height via `onHeightChanged`:

```dart
import 'package:capillary_hydra/hydra.dart';
import 'package:flutter/material.dart';

class GamificationSection extends StatefulWidget {
  const GamificationSection({super.key});
  @override
  State<GamificationSection> createState() => _GamificationSectionState();
}

class _GamificationSectionState extends State<GamificationSection> {
  double _bannerHeight = 300;

  @override
  Widget build(BuildContext context) {
    return SizedBox(
      height: _bannerHeight,
      child: GamificationBannerView(
        bannerId: 'your_banner_id',
        onHeightChanged: (height) {
          if (height > 0) setState(() => _bannerHeight = height);
        },
      ),
    );
  }
}
```

HydraCore Class APIs

`import 'package:capillary_hydra/hydra.dart';`

1. Update User

```dart
static Future<bool> updateUser({
  required HydraUserDetails userDetails,
  Map<String, dynamic>? customData,
  List<Map<String, dynamic>>? subscriptions,
});
```

Parameter	Type	Required/Optional	Description
userDetails	HydraUserDetails	required	Read more about this class here.
customData	Map<String, dynamic>?	optional	Custom attributes that can be used for analytics.
subscriptions	List<Map<String, dynamic>>?	optional	List of subscriptions, each including channel, accountId, priority, type and sourcename.

Returns: A Future which resolves to a bool indicating if the update was successfully requested.

Usage:

```
bool updateUserResult = await HydraCore.updateUser(
  userDetails: updatedUserDetails,
  customData: {"Key": "Value"},
  subscriptions: [
    {"channel": "channel1", "type": "topic1", "priority": 1},
  ],
);
```

2. Update Geo Config

```
static Future<bool> updateGeoConfig({required HydraGeoConfig config});
```

Parameter	Type	Required/Optional	Description
config	HydraGeoConfig	required	Read more about this class here.

Returns: A Future which resolves to a bool indicating if the update of geo config was successfully requested.

Usage:

```
bool updateGeoConfigResult = await HydraCore.updateGeoConfig(config: config);
```

3. Update User Push Preferences

```
static Future<bool> updatePushPreferences({
  required String customerId,
  required HydraPushPreferences preferences,
});
```

Parameter	Type	Required/Optional	Description
customerId	String	required	Customer id of the user to update push preferences.
preferences	HydraPushPreferences	required	Read more about this class here.

Returns: A Future which resolves to a bool indicating if the user push preferences update was successfully requested.

Usage:

```
bool result = await HydraCore.updatePushPreferences(
  preferences: pushPreferences,
  customerId: "8888888888",
);
```

4. Set Hydra Account Configuration (multi-account setup)

```
static Future<bool> setHydraAccountConfig({required HydraAccountConfig hydraAccountConfig});
```

Parameter	Type	Required/Optional	Description
hydraAccountConfig	HydraAccountConfig	required	Read more about this class here, including the optional `orgId` (Anonymous/Visitor Flow) and `gamificationKey` / `gamificationRegion` / `gamificationBaseUrl` (Gamification) fields.

Returns: A Future which resolves to a bool indicating whether the account config update was successfully requested. Use this when you're fetching Hydra account values yourself rather than via Firebase Remote Config.

Usage:

```
bool result = await HydraCore.setHydraAccountConfig(hydraAccountConfig: hydraAccountConfig);
```

5. Set Firebase Remote Config Keys (multi-account setup)

```
static Future<bool> setFirebaseRemoteConfigKeys({required HydraFirebaseRemoteConfigKeys
hydraFirebaseRemoteConfigKeys});
```

Parameter	Type	Required/Optional	Description
hydraFirebaseRemoteConfigKeys	HydraFirebaseRemoteConfigKeys	required	Read more about this class here, including the optional `keyForOrgId` (Anonymous/Visitor Flow) and `keyForGamificationKey` / `keyForGamificationRegion` / `keyForGamificationBaseUrl` (Gamification) key names.

Returns: A Future which resolves to a bool indicating whether the remote config keys update was successfully requested. Use this when you have multiple Hydra accounts configured under different Firebase Remote Config key names.

Usage:

```
bool result = await HydraCore.setFirebaseRemoteConfigKeys(hydraFirebaseRemoteConfigKeys:
hydraFirebaseRemoteConfigKeys);
```

Hydra Class APIs

`import 'package:capillary_hydra/hydra.dart';`

1. Identify

```
static Future<bool> identify({
  required HydraUserDetails userDetails,
  Map<String, dynamic>? customAttributes,
});
```

Parameter	Type	Required/Optional	Description
userDetails	HydraUserDetails	required	Identifies the current visitor as a known user.
customAttributes	Map<String, dynamic>?	optional	Custom attributes that can be used for analytics.

Returns: A Future which resolves to a bool indicating if identify was successfully requested.

Usage:

```
bool result = await Hydra.identify(userDetails: userDetails);
```

2. Track

```
static Future<bool> track({
  required String eventName,
  Map<String, dynamic>? attributes,
});
```

Parameter	Type	Required/Optional	Description
eventName	String	required	Name of the event you want to report.
attributes	Map<String, dynamic>?	optional	Key value pairs for additional data with the event.

Returns: A Future which resolves to a bool indicating if the event was successfully reported.

Usage:

```
bool result = await Hydra.track(eventName: 'product_viewed', attributes: {'productId': 'abc123'});
```

3. Reset

```
static Future<bool> reset({required String customerId});
```

Parameter	Type	Required/Optional	Description
customerId	String	required	Customer id of the user to reset.

Returns: A Future which resolves to a bool indicating if reset was successfully requested.

Usage:

```
bool result = await Hydra.reset(customerId: '8888888888');
```

4. Get Current User State

```
static Future<UserState?> getCurrentUserState();
```

Returns: A Future which resolves to the current UserState, or null.

Usage:

```
UserState? state = await Hydra.getCurrentUserState();
```

5. Get Visitor Details

```
static Future<HydraVisitorInfo?> getVisitorDetails();
```

Returns: A Future which resolves to a HydraVisitorInfo (`visitorId`), or null.

Usage:

```
HydraVisitorInfo? info = await Hydra.getVisitorDetails();
```

6. Is Visitor Flow Enabled

```
static Future<bool> isVisitorFlowEnabled();
```

Returns: A Future which resolves to a bool indicating whether the anonymous visitor flow is enabled for this account.

Usage:

```
bool enabled = await Hydra.isVisitorFlowEnabled();
```

7. Gamification Screen Appeared / Disappeared

```
static Future<void> gamificationScreenAppeared({required String screenName});
static Future<void> gamificationScreenDisappeared({required String screenName});
```

Parameter	Type	Required/Optional	Description
screenName	String	required	Name of the screen to track for gamification screen views. Pass the **same** `screenName` to both calls.

Usage:

```
Hydra.gamificationScreenAppeared(screenName: 'HomeScreen');
// ... later, when the screen unmounts:
Hydra.gamificationScreenDisappeared(screenName: 'HomeScreen');
```

8. Set Cap Token

```
static Future<void> setCapToken({required String token});
```

Must be called before mounting a `GamificationLoyaltyView` — see Gamification.

Usage:

```
Hydra.setCapToken(token: token);
```

HydraPushNotification class APIs

1. Get FCM Token

```
static Future<String> getFCMToken();
```

Returns: A Future which resolves to a String containing the FCM token. If the FCM token is not available, it throws an error.

Usage:

```
String fcmToken = await HydraPushNotification.getFCMToken();
```

2. Request Permission for Push Notifications

```
static Future<bool> requestPermission();
```

Returns: A Future which resolves to a bool indicating if the permission was granted. It can also throw an error.

Usage:

```
bool granted = await HydraPushNotification.requestPermission();
```

HydraInbox class APIs

1. Get list of Hydra Notifications

```
static Future<List<HydraNotification>> getInboxMessages();
```

Returns: A Future which resolves to a list of HydraNotification.

Usage:

```
List<HydraNotification> notifications = await HydraInbox.getInboxMessages();
```

2. Get Unread Hydra Notifications Count

```
static Future<int> getUnreadInboxMessagesCount();
```

Returns: A Future which resolves to a number containing the unread count.

Usage:

```
int unreadNotificationCount = await HydraInbox.getUnreadInboxMessagesCount();
```

3. Mark a Hydra Notification as Read

```
static Future<bool> markInboxMessageAsRead({required HydraNotification notification});
```

Parameter	Type	Required/Optional	Description
notification	HydraNotification	required	Hydra Notification to mark as read from the list obtained via getInboxMessages.

Returns: A Future which resolves to a bool if marking the notification as read is successfully requested.

Usage:

```
bool marked = await HydraInbox.markInboxMessageAsRead(notification: notification);
```

4. Delete a Hydra Notification

```
static Future<bool> deleteInboxMessage({required HydraNotification notification});
```

Parameter	Type	Required/Optional	Description
notification	HydraNotification	required	Hydra Notification to delete from the list obtained via getInboxMessages.

Returns: A Future which resolves to a bool if deleting the notification is successfully requested.

Usage:

```
bool deleted = await HydraInbox.deleteInboxMessage(notification: notification);
```

5. Delete all Hydra Notifications

```
static Future<bool> deleteAllInboxMessages();
```

Returns: A Future which resolves to a bool if deleting all notifications is successfully requested.

Usage:

```
bool deleted = await HydraInbox.deleteAllInboxMessages();
```

HydraInboxUI class APIs

1. Show In-Built Notification Center

```
static Future<void> showNotificationCenter();
```

Returns: A Future which resolves to void.

Usage:

```
HydraInboxUI.showNotificationCenter();
```

2. Hide In-Built Notification Center if Showing

```
static Future<void> hideNotificationCenterIfShowing();
```

Returns: A Future which resolves to void.

Usage:

```
await HydraInboxUI.hideNotificationCenterIfShowing();
```

HydraNativeEvents class APIs

To receive callbacks on various events from the native Hydra SDKs, register a handler using this class. There are functions for removing handlers too.

1. Add Event Handler

```
static void addEventHandler(HydraNativeEventHandler handler);
```

Parameter	Type	Required/Optional	Description
handler	HydraNativeEventHandler	required	An instance of a class that extends or implements HydraNativeEventHandler.

Usage:

```
HydraNativeEvents.addEventHandler(handler);
```

2. Remove Event Handler

```
static void removeEventHandler(HydraNativeEventHandler handler);
```

Parameter	Type	Required/Optional	Description
handler	HydraNativeEventHandler	required	An instance of a class that extends or implements HydraNativeEventHandler.

Usage:

```
HydraNativeEvents.removeEventHandler(handler);
```

3. Remove All Event Handlers

```
static void removeAllEventHandlers();
```

Usage:

```
HydraNativeEvents.removeAllEventHandlers();
```

HydraNativeEventHandler

Extend or implement this class and override the callbacks you're interested in (all methods have empty default bodies):

```
class HydraNativeEventHandler {
  void onTokenAvailable({required String token}) {}
  void onHydraNotificationReceived({required HydraNotification notification}) {}
  void onHydraNotificationClicked({HydraNotificationCTA? cta, HydraNotification? notification}) {}
  void onHydraNotificationDismissed({required HydraNotification notification}) {}
  void onHydraNotificationShown({required HydraNotification notification}) {}
  void onNonHydraNotificationReceived({required NonHydraNotification notification}) {}
  void onNonHydraNotificationClicked({NonHydraNotification? notification}) {}
  void onNotificationCenterDismissed() {}
  void onHydraSdkInitializationResult({required bool success, String? error}) {}
  // Anonymous / Visitor Flow:
  void onConfigFetched({required bool isVisitorFlowEnabled}) {}
  void onLegacyFlowReady() {}
  void onVisitorSessionReady() {}
  void onUserStateChanged({UserState? userState}) {}
}
```

1. **`onTokenAvailable`** — fired when a new FCM token is available; provided as a `String`.
2. **`onHydraNotificationReceived`** — fired when a Hydra notification is received; provides the `HydraNotification`.
3. **`onHydraNotificationClicked`** — fired when a Hydra notification is clicked in the system notification tray/center on either platform, when a notification is clicked in the SDK's built-in notification center, or when a click is received on an in-app message. Provides both the `HydraNotification` and the `HydraNotificationCTA` that was clicked.
4. **`onHydraNotificationDismissed`** — fired when a Hydra notification is dismissed in the system tray/center. Provides the `HydraNotification`.
5. **`onHydraNotificationShown`** — fired when a Hydra notification is shown in the system tray/center. Provides the `HydraNotification`.
6. **`onNonHydraNotificationReceived`** / **`onNonHydraNotificationClicked`** — same as above, for non-Hydra push notifications.
7. **`onNotificationCenterDismissed`** — fired when the built-in notification center is dismissed.
8. **`onHydraSdkInitializationResult`** — since SDK initialization happens automatically, use this to check whether it succeeded or failed (with an optional error).
9. **`onConfigFetched`** / **`onLegacyFlowReady`** / **`onVisitorSessionReady`** / **`onUserStateChanged`** — see Anonymous / Visitor Flow.

There's no dedicated "inbox updated" event — refresh your UI from `onHydraNotificationReceived` or `onNotificationCenterDismissed`, or by polling `HydraInbox` on demand.

HydraCore Model Classes and Enumerations

```
import 'package:capillary_hydra_platform_interface/capillary_hydra_platform_interface.dart';
```

HydraUserDetails

Used across multiple `HydraCore` APIs for signing up, signing in, and updating user details.

```
HydraUserDetails({
  required this.customerId,
  this.email,
  this.phoneNumber,
  this.firstName,
  this.lastName,
});
```

Parameter	Type	Required/Optional	Description
customerId	String	required	Can be mobile, email, InTouchUserId, externalId or FCMToken, depending on how your app is configured on the Capillary CRM dashboard (Organisation Settings → Omni Channel Settings → Channel Configurations → MAPP_SDK → Account Linking Identifier).
firstName	String?	optional	First name of the user.
lastName	String?	optional	Last name of the user.
email	String?	optional	Email of the user.
phoneNumber	String?	optional	Phone number of the user.

Usage:

```
HydraUserDetails userDetails = HydraUserDetails(
  customerId: '8888888888',
  firstName: 'FirstName',
  lastName: 'LastName',
  email: 'test@capillarytech.com',
  phoneNumber: '918888888888',
);
```

HydraGeoConfig

Used in `updateGeoConfig`.

```
HydraGeoConfig({
  required this.city,
  required this.country,
  required this.countryCode,
  this.timezone,
});
```

Parameter	Type	Required/Optional	Description
city	String	required	City of the user.
country	String	required	Country of the user.
countryCode	String	required	Country code for mobile number.
timezone	String?	optional	User's timezone.

Usage:

```
HydraGeoConfig geoConfig = HydraGeoConfig(
  city: "Bangalore",
  country: "India",
  countryCode: "IN",
  timezone: "Asia/Kolkata",
);
```

HydraPushPreferences

Used to update a user's push preferences.

```
HydraPushPreferences({
  required this.promPushEnabled,
  required this.transPushEnabled,
});
```

Parameter	Type	Required/Optional	Description
promPushEnabled	bool	required	Enable/disable promotional push notifications.
transPushEnabled	bool	required	Enable/disable transactional push notifications.

Usage:

```
HydraPushPreferences pushPreferences = HydraPushPreferences(
  promPushEnabled: false,
  transPushEnabled: true,
);
```

HydraLogLevel

```
enum HydraLogLevel {
  off(0),
  error(1),
  warn(2),
  info(3),
  debug(4),
  verbose(5);

  final int value;
}
```

HydraAccountConfig

```
class HydraAccountConfig {
  final String accountId;
  final String baseUrl;
  final String sslPublicKey;
  final HydraLogLevel remoteLogLevel;
  final String? orgId;
  final String? gamificationKey;
  final String? gamificationRegion;
  final String? gamificationBaseUrl;
}
```

Parameter	Required/Optional	Description
accountId	required	Account id from the apps team.
baseUrl	required	Base Url (End Point) from the apps team.
sslPublicKey	required	SSL public key from the apps team.
remoteLogLevel	required	See HydraLogLevel.
orgId	optional	Required only for the Anonymous/Visitor Flow.
gamificationKey	optional	Required only for Gamification.
gamificationRegion	optional	Required only for Gamification.
gamificationBaseUrl	optional	Required only for Gamification.

Usage:

```
HydraAccountConfig hydraAccountConfig = HydraAccountConfig(
  accountId: accountId,
  baseUrl: baseUrl,
  sslPublicKey: sslPublicKey,
  remoteLogLevel: HydraLogLevel.debug,
  orgId: orgId,
  gamificationKey: gamificationKey,
  gamificationRegion: gamificationRegion,
  gamificationBaseUrl: gamificationBaseUrl,
);
```

HydraFirebaseRemoteConfigKeys

```
class HydraFirebaseRemoteConfigKeys {
  final String keyForAccountId;
  final String keyForBaseUrl;
  final String keyForSsl;
  final String keyForRemoteLogLevel;
  final String? keyForOrgId;
  final String? keyForGamificationKey;
  final String? keyForGamificationRegion;
  final String? keyForGamificationBaseUrl;
}
```

Usage:

```
final hydraFirebaseRemoteConfigKeys = {
  'ind_account': HydraFirebaseRemoteConfigKeys(
    keyForAccountId: 'ind_hydra_account_id',
    keyForBaseUrl: 'ind_hydra_base_url',
    keyForSsl: 'ind_hydra_ssl_public_key',
    keyForRemoteLogLevel: 'ind_hydra_remote_log_level',
    keyForOrgId: 'ind_hydra_org_id',
    keyForGamificationKey: 'ind_hydra_gamification_key',
    keyForGamificationRegion: 'ind_hydra_gamification_region',
    keyForGamificationBaseUrl: 'ind_hydra_gamification_base_url',
  ),
};
```

UserState

```
enum UserState { none, legacyRegistered, visitor, merged, signedOut }
```

HydraVisitorInfo

```
class HydraVisitorInfo {
  final String? visitorId;
}
```

HydraNotification Model Classes and Enumerations

HydraNotification

Model class for Hydra Notification objects received from the native SDK; all fields are final.

```
class HydraNotification {
  final String? cuid;
  final String? scope;
  final String? campaignId;
  final String? variationId;
  final String? senderId;
  final String? senderSource;
  final String? custom;
  final Map<String, dynamic>? customData;
  final String? priority;
  final String? validity;
  final String? messageId;
  final HydraNotificationGateway? gateway;
  final HydraNotificationBody? body;
  final HydraNotificationExpandedDetails? expandableDetails;
  final bool? isRead;
  final String? imageIdentifier;
  final String? accountId;
  final String? channelId;
  final String? payload;
  final DateTime? createdAt;
  final bool? isExpanded;
  final bool? isMessageEllipsized;
  final bool? isTitleEllipsized;
  final bool? showArrow;
}
```

HydraNotificationBody

```
class HydraNotificationBody {
  final String? title;
  final String? message;
  final String? image;
  final HydraNotificationCTA? cta;
}
```

HydraNotificationCTA

```
class HydraNotificationCTA {
  final HydraNotificationCTAType? type;
  final String? action;
  final String? actionText;
}
```

HydraNotificationCTAType

```
enum HydraNotificationCTAType { external, deepLink, rating, feedback, appRating, unknown }
```

HydraNotificationExpandedDetails

```
class HydraNotificationExpandedDetails {
  final List<HydraNotificationCTA>? ctas;
  final String? message;
  final String? image;
  final String? style;
}
```

HydraNotificationGateway

```
enum HydraNotificationGateway { fcm, xiaomi, unknown }
```

NonHydraNotification

```
class NonHydraNotification {
  final String? title;
  final String? body;
  final String? icon;
  final String? imageUrl;
  final Map<String, dynamic> data;
}
```